Classificação & Detecção de Objetos



· Visão Computacional
· Deep Learning
· Fine-Tuning
· Arquiteturas Modernas

MSc. João Oliveira

Quem sou eu?

João “dos dados” Oliveira

Proficiência:

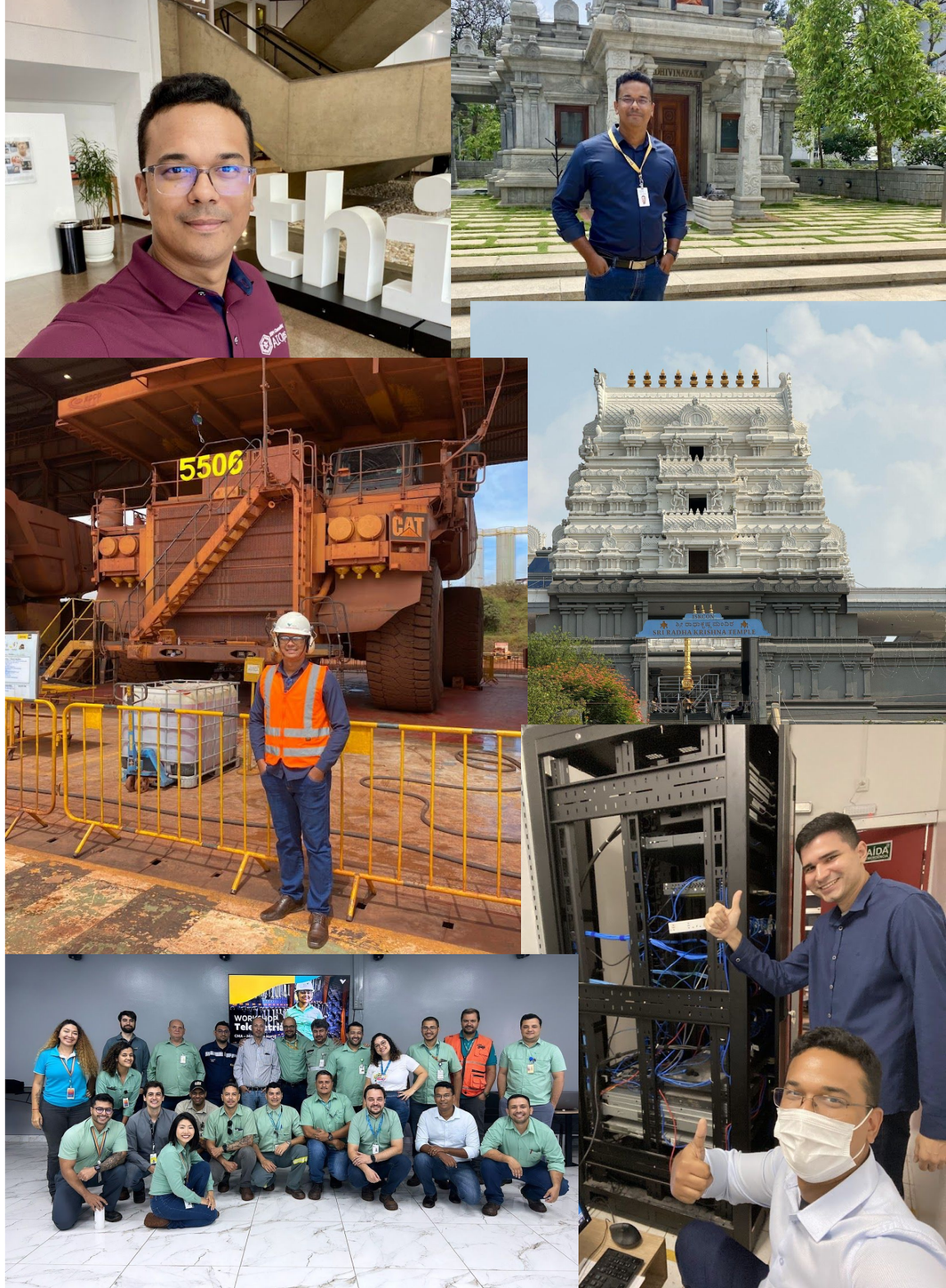
- Técnico Mecatrônico
- Bel. Sistemas de Informação
- MSc. em Ciência da Computação (UFG)
- Prof. UNIFACIMP cursos de Tecnologia da Informação
- AI Engineer – Tech Lead - IBM

L2 - Certified Data Scientist Experienced - IBM / Open Group

2x Microsoft Certified AZ-900/AI-900

1x IBM Advocate certified

Machine Learning, Visão Computacional, GenAI e AIOps



Fundamentos

Todo o material é baseado nesse livro

Szeliski.org/Book/

szeliski.org/Book/

© 2022 Richard Szeliski, The University of Washington



Welcome to the website (https://szeliski.org/Book) for the second edition of my computer vision textbook, which is now available for purchase at Amazon, Springer, and other booksellers.

To download an electronic version of the book, please fill in your information on this page. You are welcome to download the PDF website for personal use, but **not** to repost it on any other website; please post a link to this URL instead.

Note that while the content of this electronic version and the hardcopy versions are the same, the page layout is different, since the electronic version is optimized for online reading. The PDF should be enabled for commenting in your viewer. Also, sections, equations, and references are enabled. To get back to where you were, use the Previous View (Alt-Left-Arrow) command in Acrobat.

The current download count is 185241 (since 1/23/2022).

This book is largely based on the computer vision courses that I have co-taught at the University of Washington (2020, 2008, 2005, 2001) with Steve Seitz and Harpreet Sawhney and at Stanford (2003) with David Fleet.

If you're curious about the process that went into writing my book, I did an interview with Computer Vision News (March 2022).

First edition

You can still download the first edition or potentially purchase it online. The first edition is also available in Chinese and Japanese (translated by Prof. Toru Tamaki).

Slide sets and lectures

There are no official slide sets to go with the book, but please feel free to look at the University of Washington CSE 576 (Graduate Computer Vision) (2020) slides that Steve Seitz and I have put together.

Additional good sources for related courses (sorted roughly by most recent first) include:

- Noah Snavely's CS5670 - Introduction to Computer Vision class at Cornell Tech (Spring 2025)
- Bill Freeman, Antonio Torralba, and Phillip Isola's 6.8300/6.8301: Advances in Computer Vision class at MIT (Spring 2023)
- Matthew O'Toole's 16-385 Computer Vision class at CMU (Fall 2024)
- Alyosha Efros' CS194-26/294-26: Intro to Computer Vision and Computational Photography class at Berkeley (Fall 2024)
- Ioannis Gkioulekas's 15-463, 15-663, 15-862 Computational Photography class at CMU (Fall 2024)
- James Tompkin's CSCI 1430 Computer Vision class at Brown (Spring 2025)
- Yasutaka Furukawa's CMPT 412 and 762 - Computer Vision class at Simon Fraser University (Fall 2023)
- James Hays' CS 4476-A / 6476-A Computer Vision class at Georgia Tech (Fall 2022)
- Justin Johnson's EECS 498.008 / 598.008: Deep Learning for Computer Vision class at the University of Michigan (Winter 2022), which is an outstanding introduction to deep learning and visual recognition
- Yann LeCun and Alfredo Canziani's DS-GA 1008: Deep Learning class at NYU (Spring 2021)
- Luiz Velho's Fundamentals and Trends in Vision and Image Processing class at IMPA (Spring 2021)
- UC Berkeley's CS294-158-SP20: Deep Unsupervised Learning class (Spring 2020)
- Scott Wehrwein's CSCI 497P/597P - Introduction to Computer Vision class at Western Washington University (Spring 2020)
- Andrew Owens' EECS 504: Foundations of Computer Vision class at the University of Michigan (Winter 2020)

If you would like your course listed here, please contact me.

Errata

If you have any comments or feedback on the book, *please send me e-mail*. Once I have accumulated enough suggestions, I will post an updated draft with the corrections/suggestions as PDF comments. *Last updated 8/11/2025*

Motivações





Motivações





Como vocês sabem?

Como seus olhos identificam?

Motivações





Como vocês sabem?

Como seus olhos identificam?

Como a IA identifica?

MÓDULO 1 · FUNDAMENTOS

Por que CNNs? — Da Motivação à Arquitetura

A pergunta central: como fazer uma máquina "enxergar"?

O Problema com Redes Tradicionais

Redes neurais densas (MLP) precisam de um neurônio para cada pixel. Uma imagem 224×224 RGB já gera ~150.000 pesos apenas na primeira camada, levando a overfitting severo e incapacidade de reconhecer padrões locais como bordas e texturas.

A Solução: Filtros Convolucionais

CNNs usam filtros pequenos (3×3, 5×5) que se movem pela imagem (convolução), compartilhando pesos em toda a imagem. Isso permite:
• Localidade: enxergar pedaços da imagem
• Compartilhamento: menos parâmetros
• Hierarquia: bordas → texturas → partes → objetos

Hierarquia de Características (Feature Hierarchy)

Nível 1 — bordas, linhas e gradientes de pixels
Nível 2 — texturas, padrões repetitivos, orientações
Nível 3 — partes de objetos (olhos, rodas, janelas)
Nível 4 — objetos completos (gato, carro, pessoa)

Operações Fundamentais da CNN

Convolução → ReLU → Pooling → Flatten → Classificação

ReLU e Pooling

ReLU remove valores negativos, tornando a rede mais estável. Pooling reduz resolução mantendo o essencial, adicionando robustez a variações.

Tarefas de Visão Computacional

Classificação de Imagem

- Toda a imagem recebe uma única classe
- Entrada: imagem RGB
- Saída: label + probabilidade
- Ex: "gato" com 97% de confiança
- Não indica onde o objeto está

Classificação + Localização

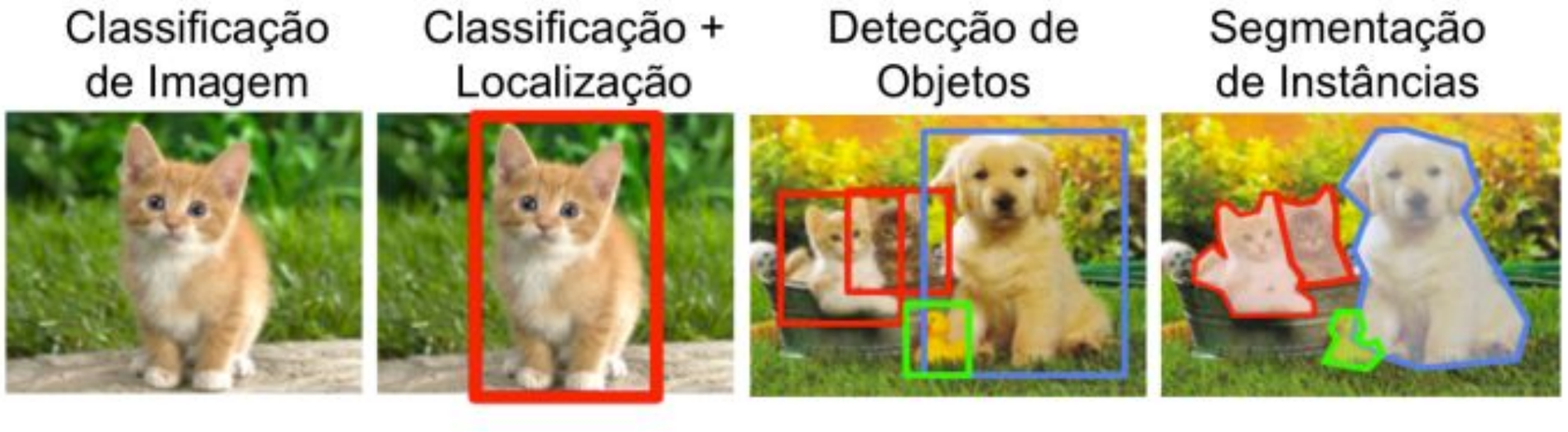
- Identifica a classe + posição
- Saída: label + 1 bounding box
- Coordenadas (x, y, largura, altura)
- Funciona apenas com 1 objeto principal
- Utilizado em problemas simples

Detecção de Objetos

- Localiza múltiplos objetos
- Saída: N classes + N bounding boxes
- Cada box com classe e confiança
- YOLO, Faster R-CNN, SSD
- Usado em câmeras, drones, varejo

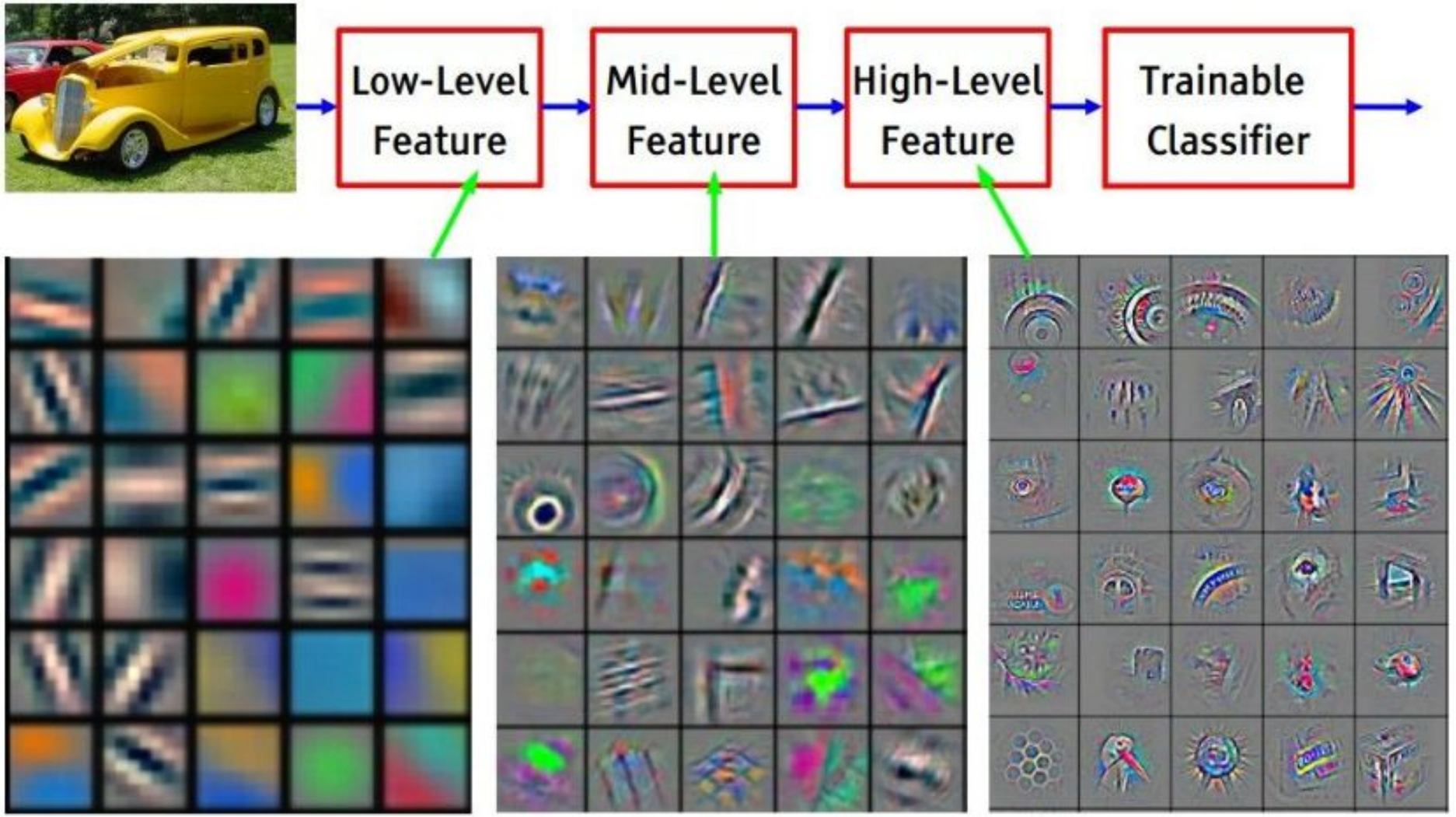
Segmentação de Instâncias

- Segmenta objetos pixel a pixel
- Cada instância tem máscara única
- Distingue objetos da mesma classe
- Mask R-CNN, SAM (Segment Anything)
- Mais preciso e computacionalmente custoso



Por que CNNs? — Da Motivação à Arquitetura

IA aprende **padrões espaciais**, assim como humanos aprendem



https://cs231n.stanford.edu/slides/winter1516_lecture7.pdf

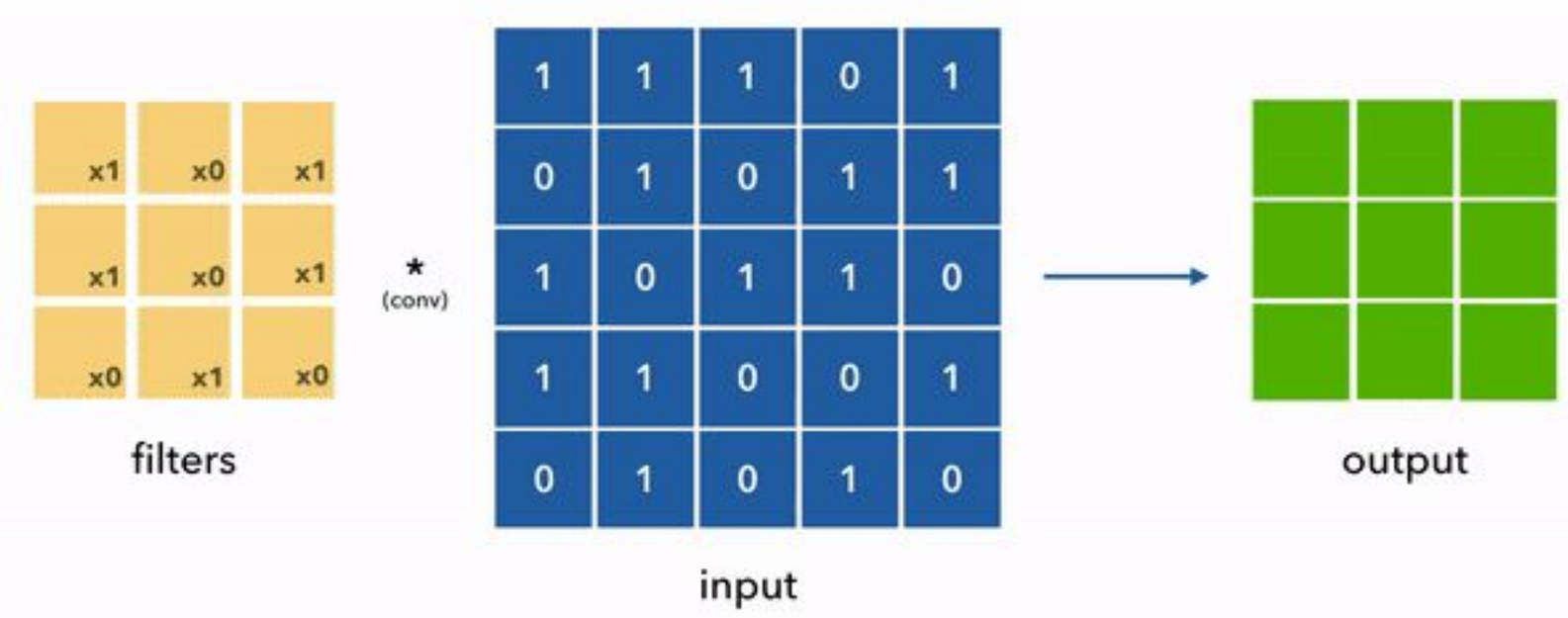
Arquitetura

Como uma CNN Transforma uma imagem?

- **Convolução**
- ReLU
- Pooling
- Flatten
- Classificação

A imagem é analisada por filtros pequenos (3×3, 5×5) que se movem sobre os pixels.

Eles identificam **bordas, texturas e formas básicas**.



Como uma CNN Transforma uma imagem?

Convolução

ReLU

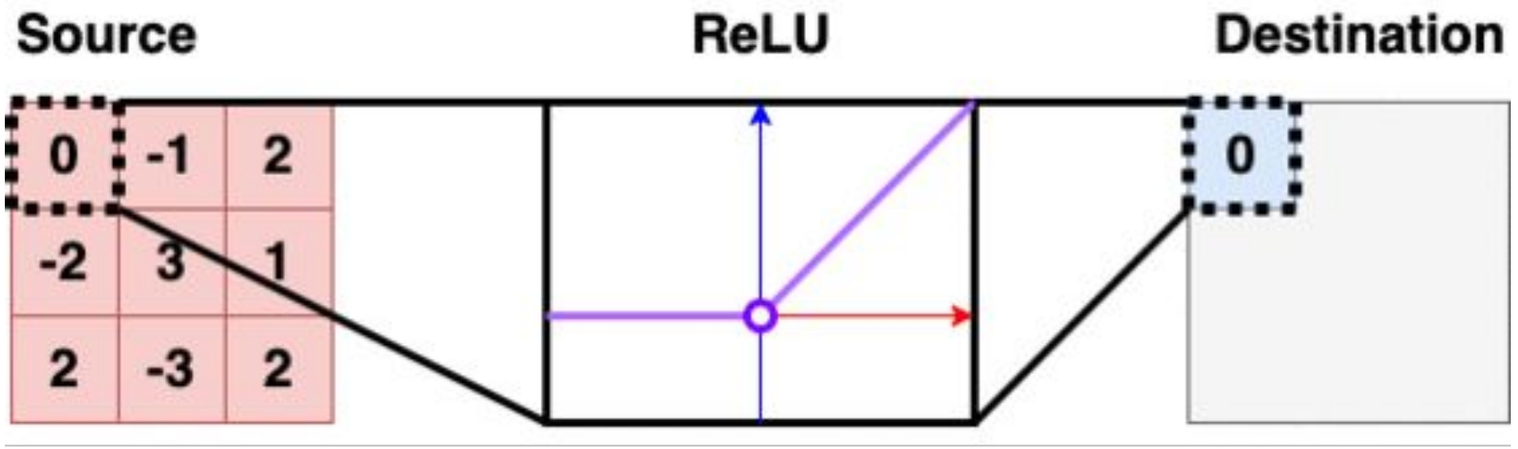
Pooling

Flatten

Classificação

A função ReLU remove valores negativos.

- Isso deixa a rede **mais rápida**, mais estável e ajuda a destacar **padrões fortes**



Como uma CNN Transforma uma imagem?

Convolução

ReLU

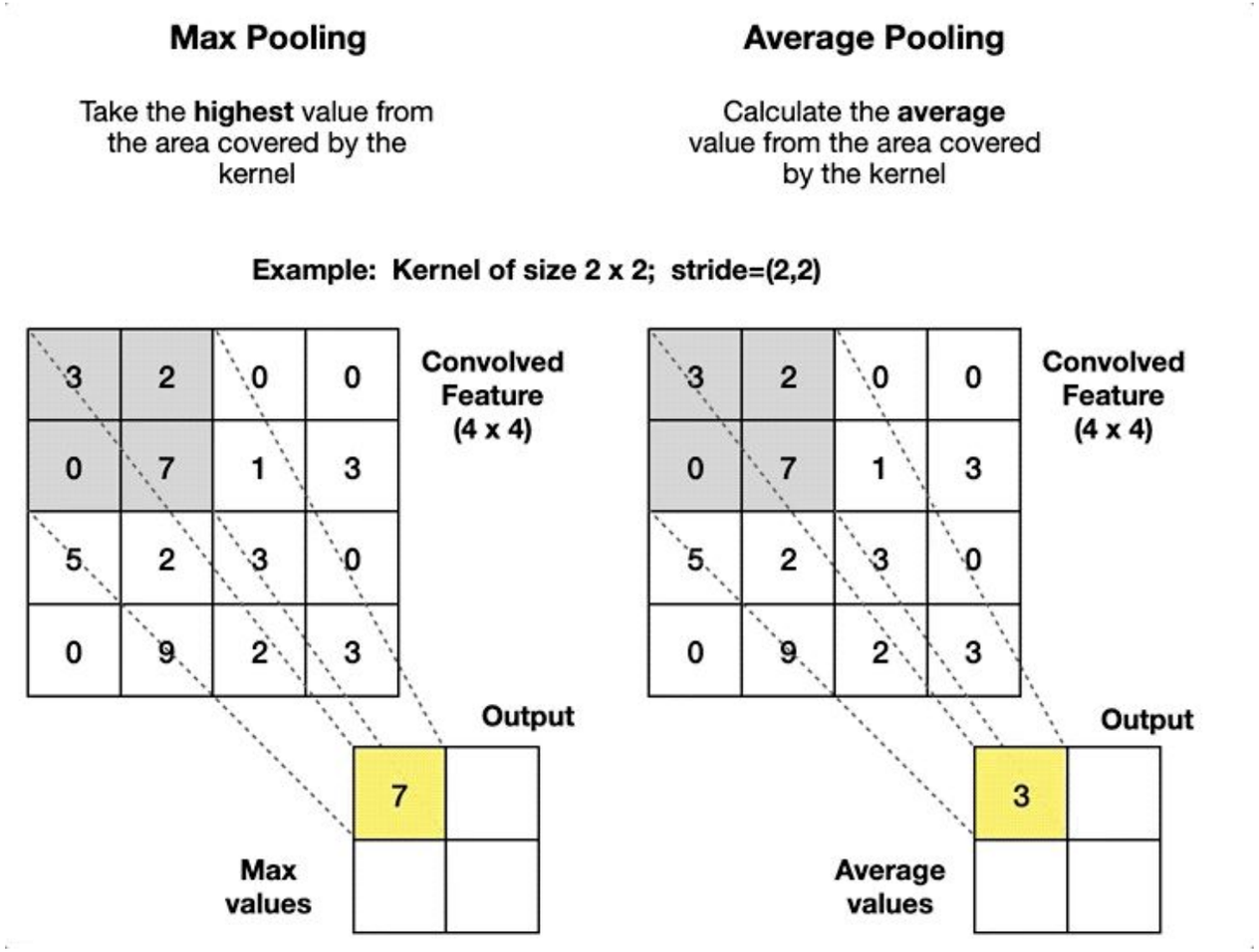
Pooling

Flatten

Classificação

A rede reduz a resolução, mantendo só o que é mais relevante.
Isso traz:

- **menos parâmetros**,
- **menos ruído**,
- **mais robustez** a variações na imagem.



Como uma CNN Transforma uma imagem?

Convolução

ReLU

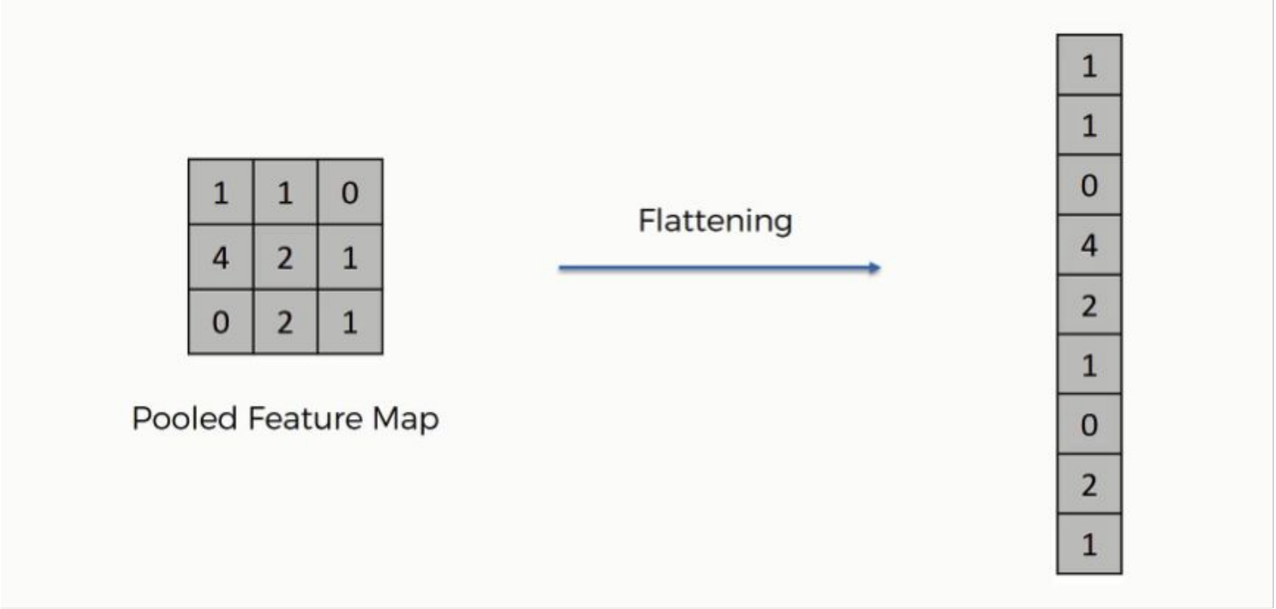
Pooling

Flatten

Classificação

Os mapas de ativação são “esticados” em um único vetor.

- É como converter as características visuais em uma **lista de sinais** para a etapa final.
- Uma matriz é convertida em lista.



Como uma CNN Transforma uma imagem?

Convolução

ReLU

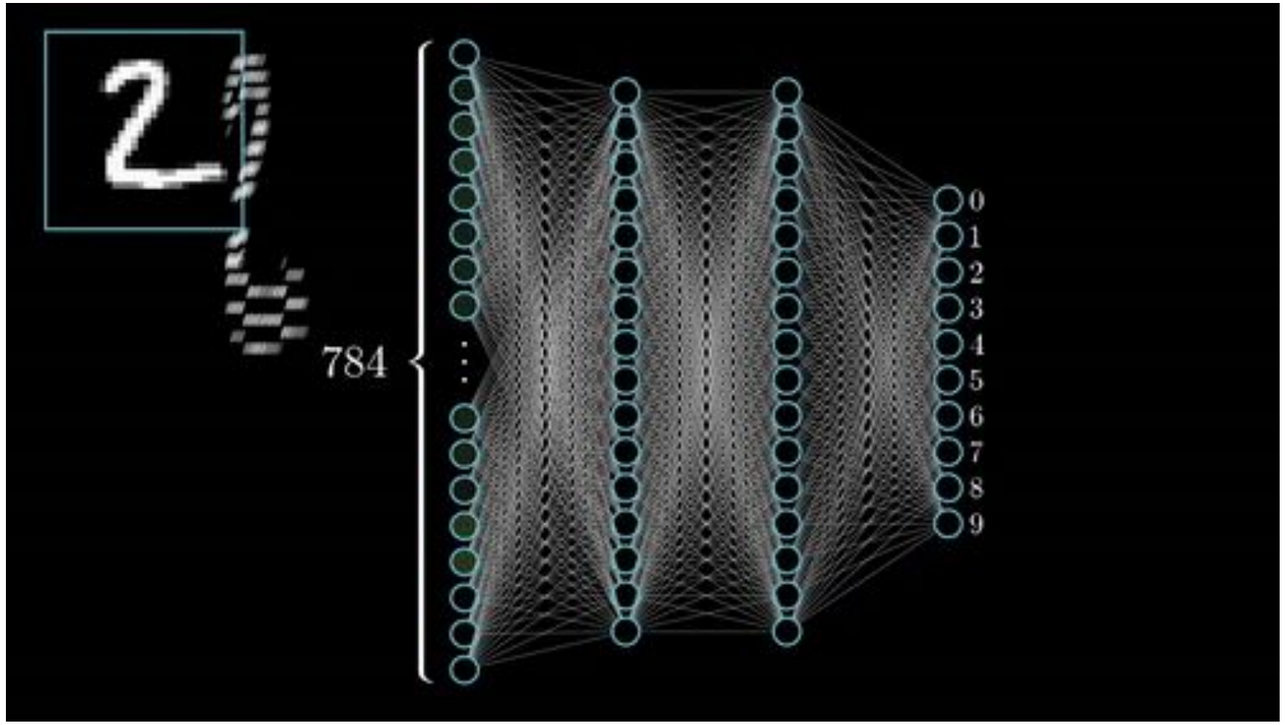
Pooling

Flatten

Classificação

Os mapas de ativação são “esticados” em um único vetor.

- É como converter as características visuais em uma **lista de sinais** para a etapa final.
- Uma matriz é convertida em lista.



CNN Explainer

https://poloclub.github.io/cnn-explainer/



What is a Convolutional Neural Network?

Modelos Pré-Treinados & Classificação

Por que não treinamos do zero?

Treinar uma CNN grande do zero é **caro**, **lento** e **exige muitos dados**.

- Precisa de **milhões de imagens** (ImageNet, COCO…)
- Computação pesada (dias/semana de GPU)
- Difícil generalizar com datasets pequenos
- Solução prática: **usar modelos que já aprenderam padrões visuais**

Principais Arquiteturas de Classificação

MobileNet (2017)	ResNet (2015)	EfficientNet (2019)	ViT (2020)
Convolução depthwise separable	Skip connections (atalhos residuais)	Compound scaling: largura + profundidade + resolução	Aplica self-attention a patches de imagem
Peso total: 4–16 MB	Resolve vanishing gradient em redes profundas	EfficientNet-B0 a B7 (crescente)	Imagem dividida em blocos 16×16 pixels
Ideal para mobile e IoT	Versões: 18, 34, 50, 101, 152 camadas	Melhor acurácia/latência do mercado	196 tokens visuais para 224×224
Muito mais rápida que VGG/ResNet	Venceu ImageNet 2015	Supera ResNet com menos FLOPs	Sem convoluções — puro Transformer
Trade-off: acurácia menor	Base de muitas arquiteturas modernas	Excelente para cloud e produção	Supera CNNs com datasets grandes

Aprendizado Transferido e ImageNet

Transfer Learning: usar um modelo pré-treinado em milhões de imagens como ponto de partida, adaptando apenas as camadas finais para a nova tarefa — sem treinar do zero.

Custo Computacional

Treinar uma ResNet-50 do zero exige dias de GPU e milhões de imagens. Com Transfer Learning, fine-tuning leva horas com poucos dados.

Generalização

Features como bordas, texturas e formas são universais. Um modelo treinado no ImageNet já sabe detectar estruturas visuais gerais.

Dados Escassos

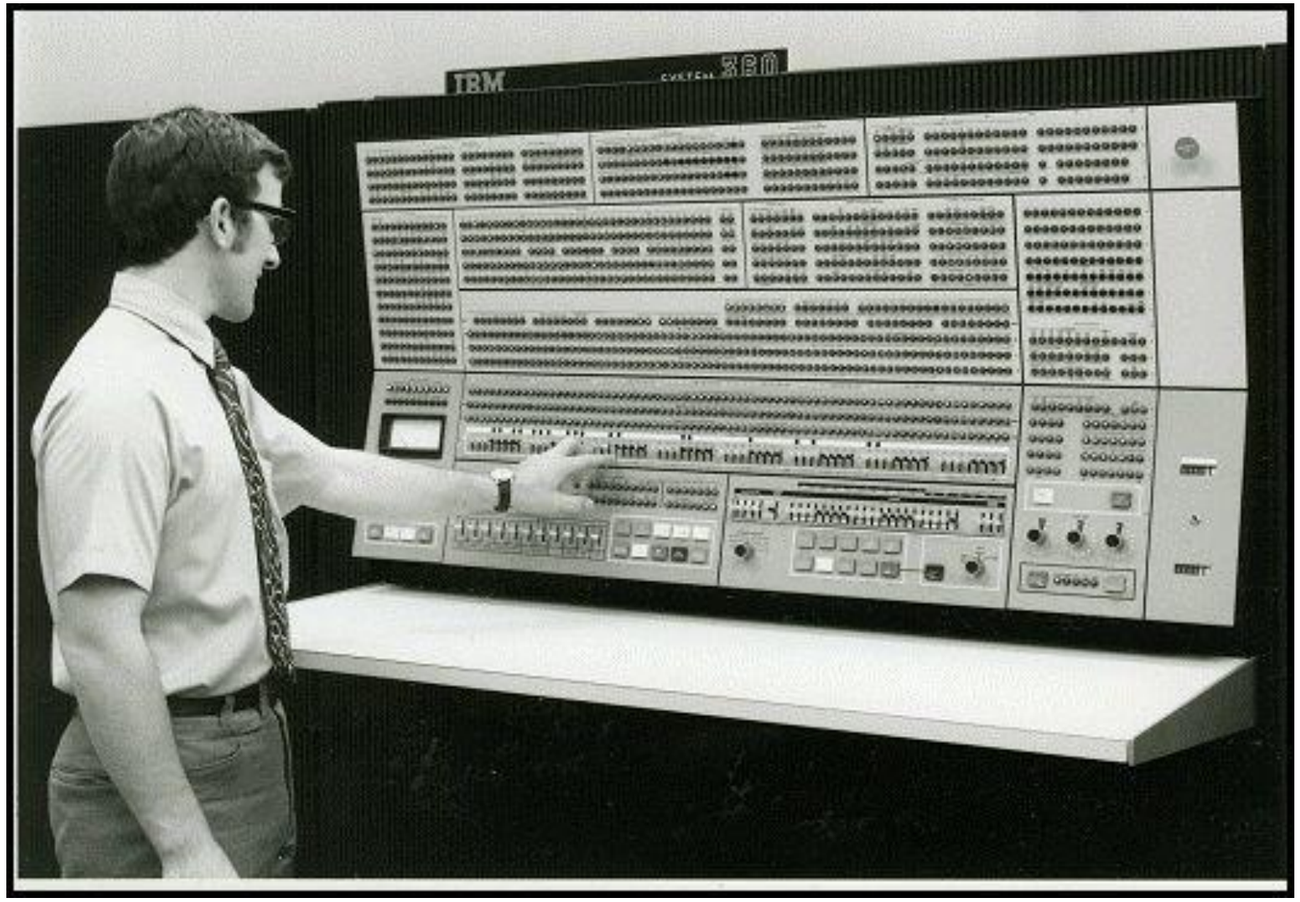
Muitos problemas reais têm poucos exemplos anotados. Transfer Learning permite resultados excelentes com apenas centenas de imagens.

ImageNet — O Dataset que Mudou Tudo

Característica	Detalhe
Tamanho	14 milhões de imagens, 21.000 classes
Competição	ILSVRC — anual desde 2010
Marco histórico	AlexNet 2012: erro de 26% → 15% em um único salto
Impacto	Início da era do Deep Learning em visão computacional

Vamos Praticar?

https://colab.research.google.com/drive/1L9cbGn3OKuR4I-2yBLctAsX7Vc4Psjhf#scrollTo=WBdhiGiXznTe



E quando não queremos apenas classificar?

Do Sliding Window ao YOLO

O Problema Original: Sliding Window

Antes do YOLO, detectar objetos exigia recortar a imagem em centenas de janelas, passar cada uma num classificador separado e combinar os resultados.

Para uma imagem típica: 1.000 posições × 5 escalas = 5.000 classificações por imagem → impraticável em tempo real.

YOLO — You Only Look Once

YOLO revolucionou a detecção ao processar a imagem inteira em uma única passada pela rede (forward pass).

- Divide a imagem em uma grade S×S
- Cada célula prediz B bounding boxes + confiança
- Cada box prediz coordenadas (x, y, w, h) + classe
- Uma única rede unificada: sem pipeline de 2 etapas

Resultado: detecção em tempo real (45-155 fps)

Vantagens sobre arquiteturas anteriores

R-CNN e Fast R-CNN usavam 2 estágios:
(1) Geração de propostas de regiões
(2) Classificação de cada proposta

YOLO faz tudo em 1 etapa:
→ Muito mais rápido
→ Menor taxa de erro em background
→ Aprende representações globais da imagem

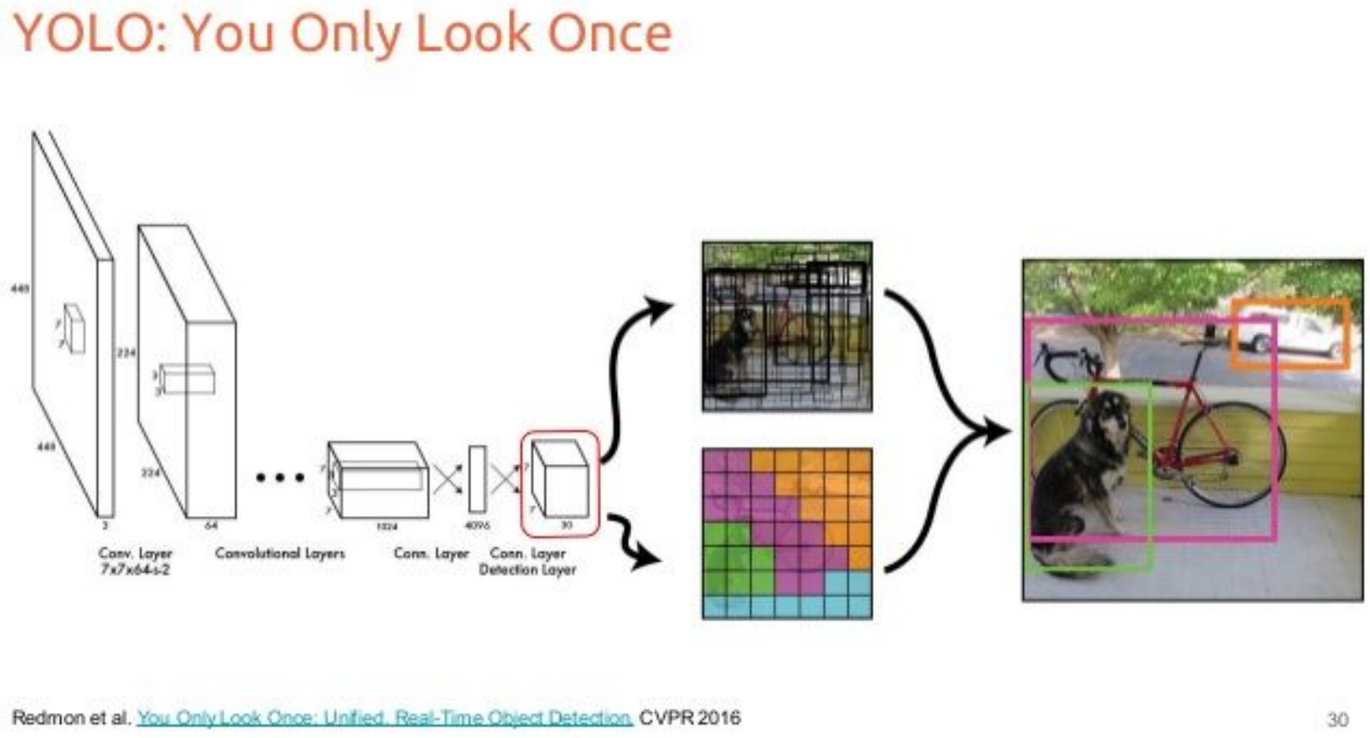
Métricas de Avaliação YOLO

mAP50 — Mean Average Precision com IoU ≥ 0.50
mAP50-95 — média de mAP de IoU 0.50 a 0.95
Precision — proporção de detecções corretas
Recall — capacidade de encontrar todos os objetos
Curvas PR — relação Precision × Recall
Confusion Matrix — erros por classe

YOLO

YOLO — You Only Look Once

- Modelo pré-treinado para **detecção de objetos em tempo real**
- Processa a imagem em **uma única passada** (por isso o nome)
- Otimizado para **velocidade + precisão**
- Usado em: câmeras, drones, carros autônomos, varejo, segurança, esportes



Famílias YOLO — Qual Versão Usar?

Modelo	Caso de Uso Ideal	Câmeras	Hardware	Observação
YOLOv8n / v12n (nano)	Mobile, Edge, IoT, drones	1–5	CPU / Jetson Nano	Alta velocidade, precisão menor
YOLOv8s / v12s (small)	Edge moderado, câmeras IP	5–20	Jetson Xavier, TPU	Equilíbrio edge/segurança
YOLOv8m / v12m (medium)	Cloud leve, batch	20–50	GPU única (T4, A10)	Mais usado em sistemas comerciais
YOLOv8l / v12l (large)	Cloud robusta, alta precisão	50–100	A100, H100, 4090	Máxima precisão acessível
YOLOv8x / v12x (xlarge)	Serviços críticos	100+	Multi-GPU	Healthcare, inspeção industrial
YOLO-World	Zero-shot detection	Variável	Cloud forte	Detecta classes não vistas no treino

Recomendação geral: YOLOv12 é o padrão atual. Para iniciantes ou projetos com poucos dados → YOLOv8 ou v10. Para mobile/edge → variante nano ou small.

MÓDULO 1 · YOLO

Fluxo Completo de Fine-Tuning YOLO

01

Escolha do Modelo

YOLOv8/9 — iniciantes
v10/11 — moderado
v12 — SOTA atual
n/s/m/l/x por hardware

02

Anotação

LabelImg / LabelMe (local)
Roboflow (online)
CVAT (equipes/avançado)
Formato: classe x y w h

03

Estrutura

train/images + labels/
val/images + labels/
data.yaml com nc e names

04

Hiperparâmetros

epochs, batch_size
img_size (640 ou 1280)
learning_rate, patience
early stopping via val

05

Treinamento

yolo train data=data.yaml
model=yolov8n.pt
monitorar loss curves
val loss × train loss

Avaliação: mAP50 · mAP50-95 · Precision · Recall · Curvas PR · Confusion Matrix — gerados automaticamente pelo Ultralytics após treino.

Vamos praticar?

https://colab.research.google.com/drive/1ynQIYJy75pFHz2RFZAtnYE9NG_L-T9lQ#scrollTo=Ba4EUOvgRar7



Hands On Fine Tunning

Ferramentas Recomendadas

1. TensorFlow/Keras

- Ideal para iniciantes
- Fácil de travar camadas e descongelar
- Bom para datasets pequenos/médios

2. PyTorch

- Mais flexível e usado em pesquisa
- Ecossistema forte (torchvision, timm)
- Ideal para quem vai treinar modelos maiores

3. HuggingFace Transformers + timm

- Para modelos modernos (ViT, ConvNeXt, SAM)
- Abstrai carregamento, métricas e datasets
- Treina em poucas linhas

4. Ultralytics / YOLO

- Fine-tuning de detectores
- Poucas linhas via CLI ou Python
- Perfeito para problemas reais de detecção

Fluxo recomendado de Fine Tuning

Escolha o modelo

Ferramenta de Anotação

Estrutura do código

Treinando o Modelo

Inferência

Escolha da versão (YOLOv8, YOLOv9, YOLOv10, YOLOv11, YOLOv12)

- **YOLOv8/9**
  Estáveis, rápidos, mais fáceis para iniciantes.
- **YOLOv10/11**
  Mais precisos, com arquitetura moderna e melhor custo-benefício entre speed × accuracy.
- **YOLOv12**
  Versão atual mais poderosa, com foco em grandes datasets e desempenho SOTA.

Qual modelo escolher?

- Se for **projeto real com poucos dados** → YOLOv8 ou YOLOv10
- Se for **muitos dados / precisão alta** → YOLOv11/12
- Se for **device fraco (mobile/edge)** → YOLOv8n ou YOLOv10n

Escolha do tamanho

- **n (nano)**: Mobile, drones, IoT
- **s (small)**: Edge, notebooks
- **m (medium)**: Projetos robustos
- **l/x (large/extra-large)**: GPU forte, máxima precisão

Fluxo recomendado de Fine Tuning

- Escolha o modelo
- Ferramenta de Anotação
- Estrutura do código
- Treinando o Modelo
- Inferência

Roboflow (mais usada)

- Interface Web super simples
- Importa qualquer dataset
- Exporta direto para YOLOv8/11/12
- Tem augmentation pronta
- Perfeito para iniciantes

LabelImg / LabelMe

- Leve, offline
- Melhor para projetos pequenos

CVAT

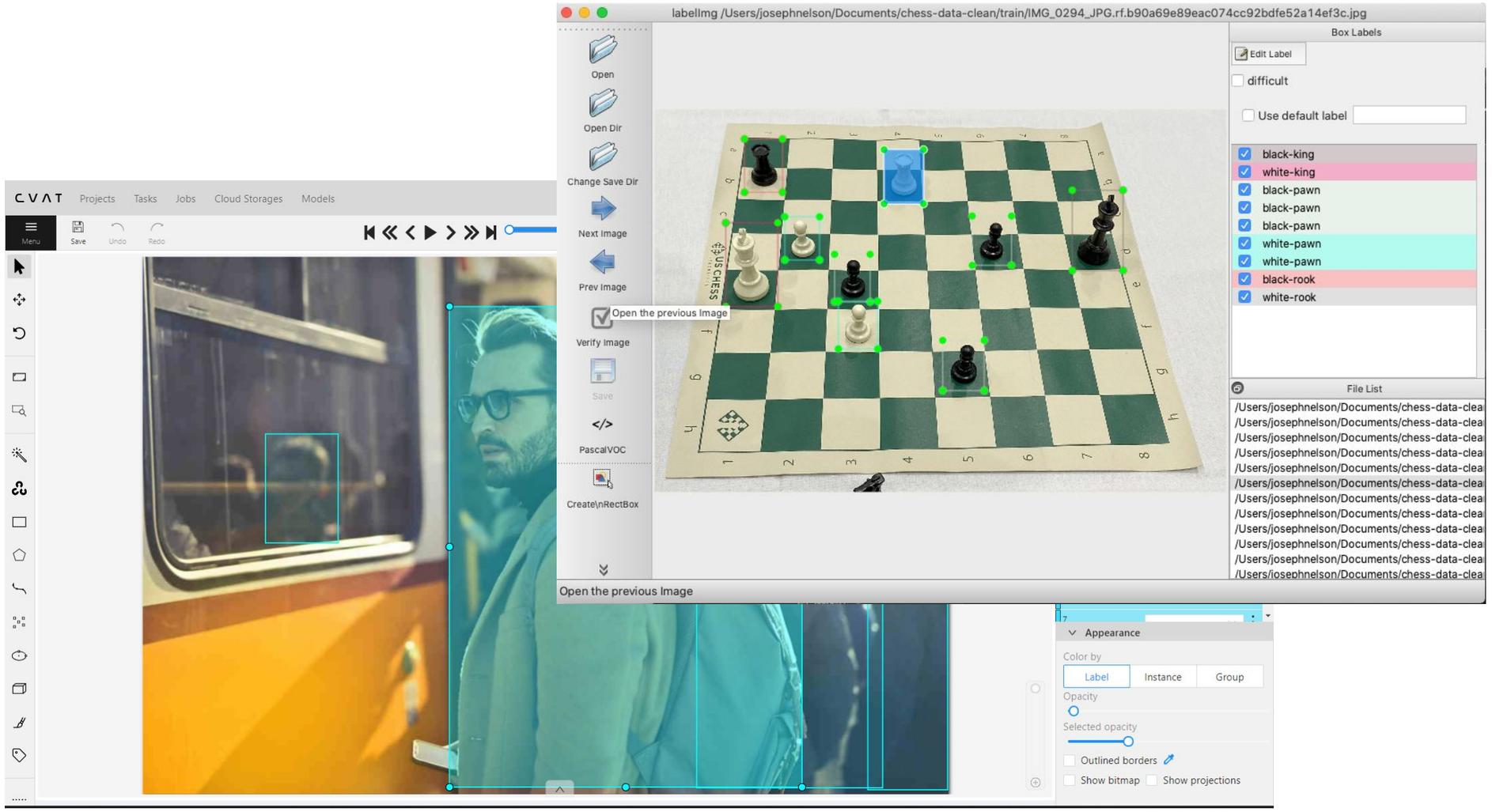
- Open-source, avançado
- Suporta times e workflows
- Ótimo para datasets grandes

MakeSense.ai

- Online, gratuito
- Simples e rápido

Fluxo recomendado de Fine Tuning

- Escolha o modelo
- **Ferramenta de Anotação**
- Estrutura do código
- Treinando o Modelo
- Inferência



Fluxo recomendado de Fine Tuning

- Escolha o modelo
- Ferramenta de Anotação
- **Estrutura do código**
- Treinando o Modelo
- Inferência

O YOLO sempre espera esta estrutura:

```kotlin
dataset/
├── train/
│   ├── images/
│   └── labels/
├── val/
│   ├── images/
│   └── labels/
└── data.yaml
```

Arquivo `data.yaml`:

```yaml
train: dataset/train/images
val: dataset/val/images
names:
  0: pessoa
  1: carro
  2: moto
```

	label_name	bbox_x	bbox_y	bbox_width	bbox_height	image_name	image_width	image_height
2	apples	94	418	304	271	apple_13.jpg	800	800
3	apples	412	259	317	308	apple_13.jpg	800	800
4	apples	208	285	244	244	apple_13.jpg	800	800
5	banana	0	166	409	590	banana_4.jpg	1100	756
6	banana	166	145	702	611	banana_4.jpg	1100	756
7	banana	247	126	340	453	banana_4.jpg	1100	756
8	apples	277	77	228	226	banana_61.jpg	640	364
9	banana	44	51	546	266	banana_61.jpg	640	364
10	apples	426	106	249	278	mixed_1.jpg	800	564
11	orange	215	60	237	271	mixed_1.jpg	800	564
12	banana	47	185	572	315	mixed_1.jpg	800	564
13	orange	71	102	452	460	orange_89.jpg	1200	900
14	orange	658	173	479	510	orange_89.jpg	1200	900
15	orange	356	10	400	488	orange_89.jpg	1200	900

Fluxo recomendado de Fine Tuning

Escolha o modelo

Ferramenta de Anotação

Estrutura do código

Treinando o Modelo

Inferência

```python
from ultralytics import YOLO

model = YOLO("yolov11s.pt")
model.train(data="data.yaml", epochs=50, imgsz=640)
```

Avaliando o Modelo

Após o treino, o YOLO gera automaticamente:

- mAP50 - é a Precisão Média calculada usando um limiar
- mAP50-95
- Precision - mede a proporção de detecções corretas (verdadeiros positivos)
- Recall - mede a capacidade de encontrar os objetos relevantes em uma imagem
- Curvas PR
- Heatmaps de erros - (via Confusion Matrix)

Fluxo recomendado de Fine Tuning

- Escolha o modelo
- Ferramenta de Anotação
- Estrutura do código
- Treinando o Modelo
- **Inferência**

```python
results = model.predict(source="pasta_de_testes/", save=True)
```

```python
results = model.predict(
    source="video_teste.mp4",
    save=True,       # salva o vídeo anotado
    stream=False     # roda tudo de uma vez
)
```

```python
for frame in model.predict(source="video.mp4", stream=True):
    boxes = frame.boxes.xyxy
    confs = frame.boxes.conf
    clss  = frame.boxes.cls
    print(boxes, confs, clss)
```

Vamos praticar?

https://colab.research.google.com/drive/1K1q7vzHdO3qFA7LnRPKtEkrbpyzJsvBU?usp=sharing



Classificação & Detecção de Objetos



- Visão Computacional
- Deep Learning
- Fine-Tuning
- Arquiteturas Modernas

MSc. João Oliveira